

1. Introduction

OOPS

- O :- Object
- O :- Oriented
- P :- Programming
- S :- Systems

WHY?

- **Encapsulation** → Bundles data + functions together, controls access
- **Abstraction** → Hides implementation details, exposes only what's needed
- **Reusability** → Code reuse via inheritance/composition
- **Modularity** → Breaks large systems into manageable objects
- **Maintainability** → Easier to modify and debug structured code
- **Scalability** → Handles growing complexity better
- **Real-world modeling** → Maps code to real-world entities clearly

Drawbacks of Functional / Procedural Programming

- Does not model **real world problem** very well.
- If a new **data is added**, all the functions needs to be **modified** to access the Data.
- Global data is accessible to all the functions.
- No **clear boundaries** and well definition of code.
- **No Modularity**: Functional programs can become monolithic and difficult to maintain as they grow in complexity.

👉 2. Intro to Class